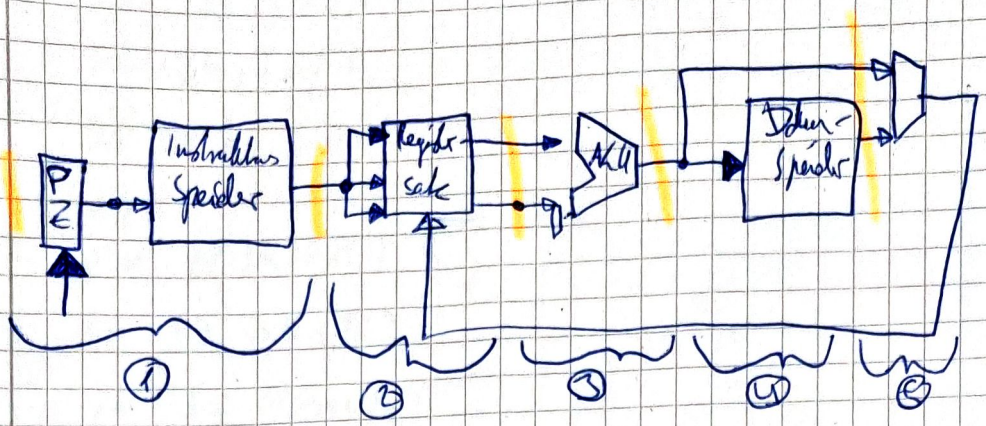
# Technische Informatik

## MIPS




### ① Instruktion fetch (IF)

→ Programmzähler speichert und liefert die aktuelle auszuführende Instruktionsadresse an den Instruktionsspeicher und das Addierwerk übermittelt die Folgeadresse

→ Instruktionsspeicher liefert die auszuführende Maschineninstruktion

### ② Instruktion Decode (ID)

Bekommt R1 und R2 und Ziel R gibt Werte von R1 und R2 an ALU weiter. Ergebnis wird später in Ziel R gespeichert

### ③ Ausführungsphase (EX)

ALU berechnet Maschinenbefehl.
Gibt Daten / Adressen weiter.

### ④ Memory Access

Quellregister speichert falls store, Zielregister kopiert [illegible] Wert des Vorgängers

### ⑤ Register zurückschreiben (Write Back)

Wert in Zielregister schreiben

# Technische Informatik Blatt 9

## 9-1 MIPS-Pipeline

a)
```
addi $2, $0, 0x42
sub  $4, $6, $1
sll  $5, $6, 3
NOP
xori $8, $4, 0xFF
lw   $9, 0x0 ($2)
sw   $5, 0x200 ($2)
and  $4, $4, $3
```

| Takt | IF | ID | EX | MEM | WB |
|---|---|---|---|---|---|
| 1 | addi | | | | |
| 2 | sub | addi | | | |
| 3 | sll | sub | addi | | |
| 4 | NOP | sll | sub | addi | |
| 5 | xori | NOP | sll | sub | addi |
| 6 | lw | xori | NOP | sll | sub |
| 7 | sw | lw | xori | NOP | sll |
| 8 | and | sw | lw | xori | NOP |
| 9 | | and | sw | lw | xori |
| 10 | | | and | sw | lw |
| 11 | | | | and | sw |
| 12 | | | | | and |

Bei Read-After-Write muss B1 schon in WB sein während B2 in ID sein muss

Vor der xori-Instruktion muss ein NOP eingefügt werden, damit das Register $4 das Ergebnis von sub $4, $6, $1 enthält und es zu keiner Datenhaftigkeit kommt. Die sub-Instruktion schreibt während der WB-Phase das Ergebnis der Subtraktion bei der steigenden Taktflanke in Register $4. Die xori-Instruktion liest den Wert aus Register $4 bei der fallenden Flanke in der ID-Phase.

b) Die xori-Instruktion liest $4 welches zuvor von sub geschrieben wurde. Die xori-Instruktion ist daher von Read-After-Write Datenabhängig

# Technische Informatik – Blatt 9

## (9-2) Programmoptimierung

```
mull $3, $2, $1
divi $3, $3, $4
mult $6, $1, $1
addi $7, $0, 13
mult $6, $6, $7
subi $3, $3, $6
mult $8, $1, $1
mult $8, $8, $1
divi $8, $5, $8
add  $3, $3, $8
```

$$\$3 = \frac{\$2 \cdot \$1}{\$4} - 13 \cdot \$1^2 + \frac{\$5}{\$13}$$

a) NOPs ohne Data Forwarding & ohne Hazard-Detection = ohne Logik

↳ = Bypass WB/Mem zu EX

```
mull $3, $2, $1
NOP
NOP
divi $3, $3, $4
mult $6, $1, $1
addi $7, $0, 13
NOP
mult $6, $6, $7
NOP
NOP
subi $3, $3, $6
mult $8, $1, $1
NOP
NOP
mult $8, $8, $1
NOP
NOP
divi $8, $5, $8
NOP
NOP
add  $3, $3, $8
```

Technische Informatik – Blatt 9

(9.2)

5) Optimierung: Ohne Data-Forwarding aber mit Hazard Detection

| | | |
|---|---|---|
| 1 | MULT | $8, $1, $1 |
| 2 | MULT | $3, $2, $1 |
| 3 | mul | $6, $1, $1 |
| 4 | addi | $7, $0, 13 |
| 5 | div | $3, $3, $4 |
| 6 | ~~mul~~ | ~~$6, $6, $1~~ |
| | ~~Mul~~ | ~~$8, $1, $1~~ |
| 7 | mul | $6, $6, $7 |
| 8 | NOP | |
| 9 | div | $8, $5, $E |
| 10 | NOP | |
| 11 | NOP | |
| 12 | add | $3, $3, $8 |

# Technische Informatik – Blatt 9

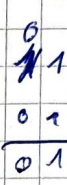
(9-3)

a) Worin besteht der Unterschied von Stalls zu einem Flush in einer MIPS-Pipeline?

**Stalls:** Halten die Pipeline ein oder mehrere Zyklen lang an (durch Einfügen von NOPs / Bubbles)
Beispielsweise notwendig um Datenkonsistenz zu erhalten, wenn Datenkonflikte bestehen.

**Pipeline-Flush:** Umwandeln von bereits in die Pipeline geladenen Instruktionen in NOPs, also effektives Entfernen von Befehlen aus der Pipeline bzw. Leerlaufenlassen der Pipeline

```
add $6, $0, $0   // $6 = 0+8
addi $4, $0, 0x7 // $4 = 7
ori  $3, $4, 0x4 // $3 = 4
loop:
lw   $2, 0x3E8($3) // $2 = 4
add  $6, $3, $6    // $6 = 4
addi $5, $4, 1     // $5 = 8
sub  $1, $2, $6    // $1 = 4-4
bne  $5, $6, loop  // $5 != $6
sw   $6, 0x400($1)
```



# Technische Informatik – Blatt 9

(9-3)

5) Simulieren Sie den Ablauf des gegebenen MIPS-Programm. Nehmen Sie für diese Teilaufgabe an, dass eine Hazard-Detection-Einheit die Stalls bei einem Konflikt einfügt – keine Data-Forwarding-Hardware gelt mir darum aus dass der Sprung nicht ausgeführt wird, Falls doch ausgeführt werden soll → NOP

| Takt | IF | ID | EX | MEM | WB |
|---|---|---|---|---|---|
| 1 | add | | | | |
| 2 | addi | add | | | |
| 3 | andi | addi | add | | |
| 4 | lw | andi | addi | add | |
| 5 | lw | andi | NOP | addi | add |
| 6 | lw | andi | NOP | NOP | addi |
| 7 | add | lw | andi | NOP | NOP |
| 8 | add | lw | NOP | addi | NOP |
| 9 | add | lw | NOP | NOP | andi |
| 10 | addi | add | lw | NOP | NOP |
| 11 | sub | addi | add | lw | NOP |
| 12 | bne | sub | addi | add | lw |
| 13 | sw | bne | ~~sub~~ | addi | andi |
| 14 | sw | bne | NOP | sub | addi |
| 15 | | sw | bne | NOP | sub |
| 16 | | | sw | bne | NOP |
| 17 | | | | sw | bne |
| 18 | | | | | |
| 19 | | | | | |
| 20 | | | | | |
| 21 | | | | | |
| 22 | | | | | |
| 23 | | | | | |
| 24 | | | | | |
| 25 | | | | | |

ID
↓
bis
in WB

bne befehl
entscheidet in ID

(9-3)

b)

| Takt | IF | ID | EX | MEM | WB |
|---|---|---|---|---|---|
| 1 | add | | | | |
| 2 | addi | add | | | |
| 3 | and | addi | add | | |
| 4 | lw | and | addi | add | |
| 5 | lw | and | NOP | addi | add |
| 6 | lw | and | NOP | NOP | addi |
| 7 | add | lw | and | NOP | NOP |
| 8 | add | lw | NOP | and | NOP |
| 9 | add | lw | NOP | NOP | and |
| 10 | addi | add | lw | NOP | NOP |
| 11 | sub | addi | add | lw | NOP |
| 12 | bne | sub | addi | add | lw |
| 13 | bne | sw | NOP | addi | add |
| 14 | sw | bne | sw | NOP | addi |
| 15 | lw | NOP | bne | sw | NOP |
| 16 | add | lw | NOP | bne | sw |
| 17 | addi | add | lw | NOP | bne |
| 18 | sub | addi | add | lw | NOP |
| 19 | bne | sub | addi | add | lw |
| 20 | bne | sub | NOP | addi | add |
| 21 | sw | bne | sw | NOP | addi |
| 22 | | sw | bne | sw | NOP |
| 23 | | lw | ~~NOP~~ | bne | NOP |
| 24 | | | sw | ~~NOP~~ | bne |
| 25 | | | | sw | NOP |
| 26 | | | | | sw |

bne wird in ID ausgeführt

→ Hazard-Detection: Einfügen von NOPs

→ bne wird in ID ausgeführt (Allgemein Jumpbefehle)

→ Bei RAW muss BZ in ID warten bis B1 in WB ist

Technische Informatik – Blatt 9

(A-3)

c) Forwarding-Fehler:

Bypässe: Forwarding ist möglich aus der MEM- und WB-Phase in die EX-Phase

| Takt | IF | ID | EX | MEM | WB |
|---|---|---|---|---|---|
| 1 | add | | | | |
| 2 | addi | add | | | |
| 3 | andi | addi | add | | |
| 4 | lw | andi | addi | add | |
| 5 | add | lw | andi | addi | add |
| 6 | addi | add | lw | andi | addi |
| 7 | sub | addi | add | lw | andi |
| 8 | bne | sub | addi | add | lw |
| 9 | sw | bne | sub | addi | add |
| 10 | sw | bne | NOP | sub | addi |
| 11 | lw | NOP | bne | NOP | sub |
| 12 | add | lw | NOP | bne | NOP |
| 13 | addi | add | lw | NOP | bne |
| 14 | sub | addi | add | lw | NOP |
| 15 | bne | sub | addi | add | lw |
| 16 | sw | bne | sub | addi | add |
| 17 | sw | bne | NOP | sub | addi |
| 18 | | sw | bne | NOP | sub |
| 19 | | | sw | bne | NOP |
| 20 | | | | sw | bne |
| | | | | | sw |

bne (Jump-Befehle) wird in ID ausgeführt

deswegen läuft der Bypass zum Execute nicht.

→ NOP wird eingefügt

~~Datenforwarding~~

Dataforwarding

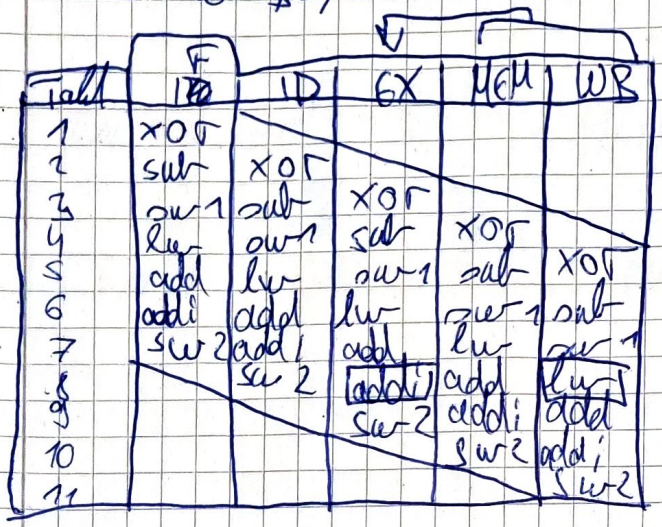
Bypässe erzeugen!

# Verschiedene Informatik - Übungsaufgaben

(7)

```
xor  $4, $5, $5
sub  $3, $2, $8
sw1: sw   $8, 80($3)
     lw   $9, 80($3)
     add  $3, $5, $8
     addi $4, $9, 20
sw2: sw   $5, 20($3)
```

Hazard Detection & Forwarding (Bypass (10B & MEM) an EX



| Takt | F / ID | ID | EX | MEM | WB |
|---|---|---|---|---|---|
| 1 | xor | | | | |
| 2 | sub | xor | | | |
| 3 | sw1 | sub | xor | | |
| 4 | lw | sw1 | sub | xor | |
| 5 | add | lw | sw1 | sub | xor |
| 6 | addi | add | lw | sw1 | sub |
| 7 | sw2 | addi | add | lw | sw1 |
| 8 | | sw2 | addi | add | lw |
| 9 | | | sw2 | addi | add |
| 10 | | | | sw2 | addi |
| 11 | | | | | sw2 |

→ Ist es ein Problem, dass der lw- & sw-Befehl die gleiche Adresse benutzt?

Nein

→ Wieso ist von der addi-Instruktion kein NOP nötig, obwohl doch das von lw-Befehl zuvor beschriebene Reg [9] benutzt werden muss?

⇒ Das Register wird in der WB-Phase von der lw-Instruktion geschrieben. Gleichzeitig ist Befehl addi in der EX-Phase. Der von lw geholte Wert kann gefahrenlos und direkt benutzt werden.

# Technische Informatik

## CRC

**Nachrichtenpolynom:**

$$G_1(x) = \sum_{i=0}^{n-1} a_i * 2^i$$

↳ Generator Polynom

$$F(x) = G_1(x) \circ x^{n-k} + R(x) = P(x) * Q(x)$$

↳ Prüfbits

↳ Rest Nachricht

$$R(x) = G_1(x) * x^{n-k} - P(x) * Q(x)$$

↳ Bzw $G_1(x) * x^{n-k} : P(x) = Q(x)$ R $R(x)$

# Technische Informatik – Blatt 10

1) Byp:

G = 1 1 1 1 1 1 0 ist CRC-5

P = CRC-5 = $\underbrace{1\,0\,0\,1\,0\,1}_{R}$

$(1\,1\,1\,1\,1\,1\,0\,\underbrace{0\,0\,0\,0\,0}_{R-1}) : (\underbrace{1\,0\,0\,1\,0\,1}_{R}) =$

---

Nachricht Fehlerprüfung ⇒ H : P

↳ Rest ⇒ Fehler

↳ kein Rest ⇒ kein Fehler oder Fehler nicht erkannt

---

3) RAW-Abhängigkeit

B1: add $12, $3, $3

B2: sub $4, $5, $12

↳ B1 stellt Daten bereit, die B2 benötigt ⇒ B2 von B1 RAW abhängig

WAR-Abhängigkeit

B3: add $4, $5, $12

B4: and $12, $6, $7

↳ B4 beschreibt eine Zelle die von B3 gelesen wird

↳ B3 von B4 WAR-Abhängig

WAW-Abhängigkeit

B5: add $12, $2, $3

B6: or $12, $5, $4

↳ B5 und B6 beschreiben selbe Zelle

↳ B5 & B6 WAW abhängig

# Technische Informatik – Blatt 10

## 10-1 CRC

a) Was bedeutet CRC und wozu dient es?

CRC (Cyclic Redundancy Check) ist ein Verfahren zur Fehlererkennung, bei welchem die zu übertragenden Daten als Polynom betrachtet werden. Für die Fehlererkennung werden sog. genannte Generatorpolynome benutzt. Das eigentliche Verfahren beruht dann auf der Polynomdivision. Bei der Rechnung wird nicht tatsächlich dividiert, sondern XOR gerechnet.

b) Berechnen Sie die Prüfbits R, wenn der Nachricht G wie gegeben sie die daraus resultierende zu übertragende Nachricht F an. Nehmen Sie als Generator-Polynom das für USB übliche CRC-5 an.

$G = 1100111011$

CRC-5 = 100101

$\hookrightarrow n = 6$

$6-1 = n-1 = 5$

(1100111011 00000) : (100101) =

```
1100111011 00000 | 100101
100101           -----------
 101101          01000
 100101          
 -------
  01000 0
  00000 0
  -------
   1000 01
   100101
   ------
    00100 1
    00000 0
    -------
     0100 10
     100101
     ------
      00001 0
      00000 0
      -------
       0001 00
       000000
       ------
        00100 0
```

$\Rightarrow R = 01000$

$F = \underbrace{1100111011}_{G}\ \underbrace{01000}_{R}$

# Technische Informatik – Blatt 10

**10-1**

c) Überprüfen Sie die folgende über USB empfangene Nachricht H auf Korrektheit. Falls bei der Übertragung ein Fehler aufgetreten ist, geben Sie an, um was für ein Fehler es sich handeln könnte.

H = 1111100110101111

CRC-5 = 100101

(1111100110101111) : (100101)

100101
11011 0
100101
10011 1
100101
10101 6
100101
11111 1
100101
11010 1
100101
10000 ⇒ Fehler erkannt!

Welcher Fehler genau geschehen ist, kann man nicht nachvollziehen. Man weiß aber auf Grund des Generatorpolynoms P, dass alle Einzelfehler und ungerade Anzahl an Fehlern, die z.B. von Burst-Fehlern in der Länge $b < 6$ von CRC-5 auf jeden Fall erkannt wird, wenn der Fehlerpolynom kein Vielfaches des Generatorpolynoms ist. Ansonsten erkennt CRC-5 Burst-Fehler der Länge ...

- $b = 6$ mit 93,750%
- $b > 6$ mit 96,875%

Jedes Generator-Polynom des (1+x) als Faktor enthält findet jede ungerade Anzahl an Fehlern.

# Technische Informatik – Blatt 10

**Burst-Fehler:** Fehler oder

z.B.: $E(x) = 1\underbrace{0011}_{5}0\,60$

↳ Burst-Fehler mit $b = 5$

**Satz 3:**

Mit einem Generator-Polynom $P(x)$ vom Grad $n-k$ erkennt man alle Burst-Fehler der Länge $b \leq n-k$ wenn $P(x)$, eben Term $x$ nicht als Faktor enthält

---

d) Überprüfen Sie die folgende empfangene Nachricht auf Korrektheit.
Als Generator-Polynom wurde hier CRC-7 verwendet

$H = 11011001011100110$

$CRC\text{-}7 = \underbrace{10001001}_{n=8}$

$(11011001011100110) : (10001001)$

$10001001$
$\;\;10100000$
$\;\;10001001$
$\;\;\;\;10100111$
$\;\;\;\;10001001$
$\;\;\;\;\;\;10111010$
$\;\;\;\;\;\;10001001$
$\;\;\;\;\;\;11001101$
$\;\;\;\;\;\;10001001$
$\;\;\;\;\;\;\;\;10001001$
$\;\;\;\;\;\;\;\;10001001$
$\;\;\;\;\;\;\;\;00000000$

→ Kein Fehler oder Nicht Erkennbarer Fehler

z.B.: Burst-Fehler der Länge $\geq 8$

# Technische Informatik B/dll/10

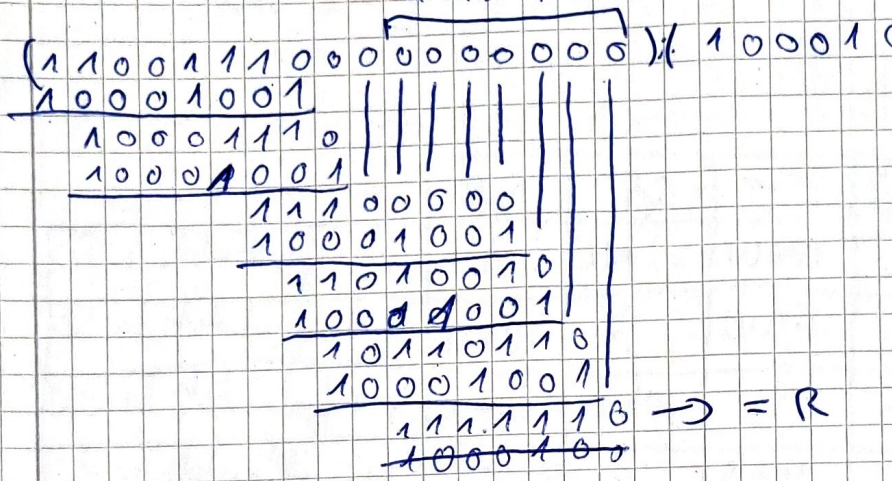
(10-1)

e) Berechnen Sie die zu übertragende Nachricht F an der Stelle G mit dem Generator-Polynom von CRC-7

G = 1100111000

CRC-7 = 10001001

n-1 = 7




F = G ∘ R = 1100111000 | 1111110

G | R

# Technische Informatik – Blatt 10

(10-3)

B1: add $2, $3, $3
B2: sub $3, $5, $2
B3: add $2, $5, $3
B4: xor $9, $2, $8
B5: add $3, $2, $9

| | B1 | B2 | B3 | B4 | B5 |
|---|---|---|---|---|---|
| B1 ist datenabhängig von | – | WAR | WAW | ~~–~~ | WAR? |
| B2 ist datenabhängig von | RAW | – | WAR | – | WAW |
| B3 ist datenabhängig von | WAW | RAW | – | – | WAR |
| B4 ist datenabhängig von | ~~RAW~~ | – | RAW | – | – |
| B5 ist datenabhängig von | ~~RAW~~ | WAW | RAW | RAW | – |

RAW nur bis zum ersten nicht bis zu dem der davor geschrieben hat
WAR zu jedem der danach schreibt

# Technische Informatik – Übungsaufgabe

K7 #65 a 321

1 0 0 0 1 0 0 1

## (11) CRC

a) Mit welchen technischen einfachen Verfahren kann ein Empfänger die bei ihm eingegangene Nachricht erkennen, wenn CRC-7 verwendet wurde aber die Prüfziffern gar nicht zum Einsatz kommen sollen?

Länge (CRC-7) = 8 = n

~~Rest~~ eines 1 Bit = 1 = k

Empfänger schickt empfangene Nachricht n - k = 8 - 1 = 7 Bit nach rechts beim schieben nach 7 Bit ab

b) Geben Sie an, wie man bei CRC das Nachrichtpolynom zur Basis x einer in k Bits kodierten Nachricht darstellt

$$G(x) = \sum_{i=0}^{k-1} a_i \cdot x^i$$

c) Wie berechnet man die einer Nachricht angefügten Prüfziffern?

~~R(x) = G(x) / P(x)~~ G(x) / P(x) = ℰ R R(x)

Das Prüfbit-Polynom R(x) ist der Rest der Polynomdivision von G(x) (nach links verschoben um n - k Stellen) durch ein Generator-polynom P(x).

# Technische Informatik – Klausuraufgabe

(10)

100101

d) Bei CRC-5 = $x^5 + x^2 + 1$. Bestimmen Sie anhand einer Berechnung den tatsächlich zu sendenden Bitstrom.

(1000101011000008) ( 100101 )

```
1000101011000008 | 100101
100101           |
 11011           |
 100101          |
  110001         |
  100101         |
   101000        |
   100101        |
    110100       |
    100101       |
     100001      |
     100101      |
      00110      |
```

→ R = 00110

F = GOR =

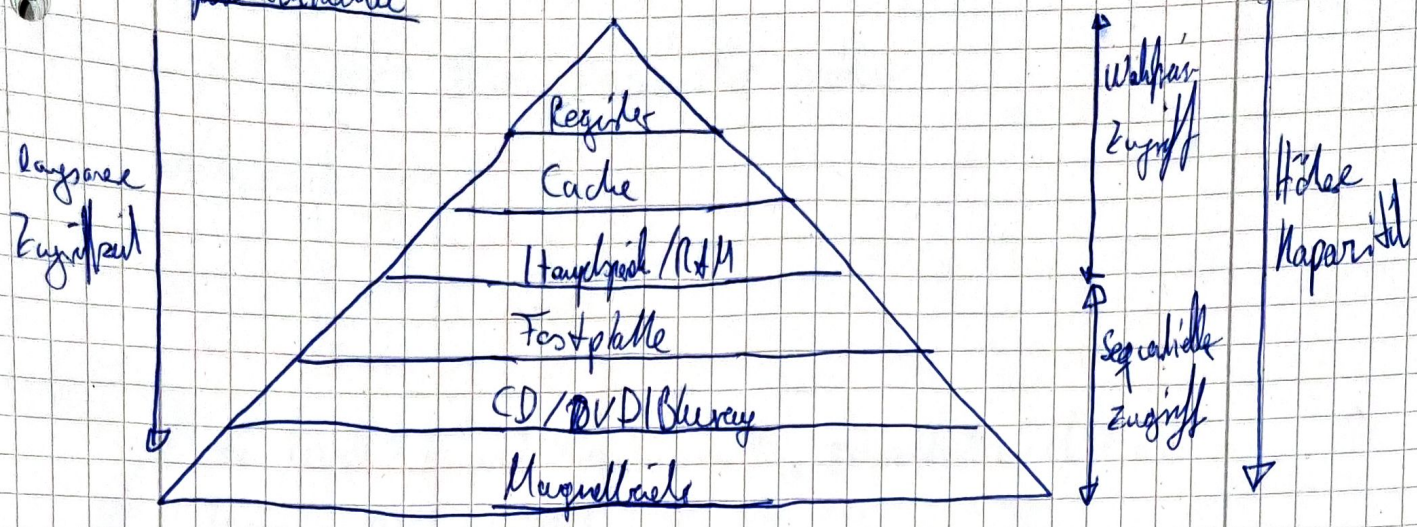
100010101101110

e) Wie kann man bei einer mit CRC gesicherten Nachricht feststellen, dass ein fehlendes Bit bei der Übertragung verloren ging?

Gar nicht

Technische Informatik

## Speicherhierarchie




1 KByte = 1024 Bytes
1 MByte = 1024 KB
1 GByte = 1024 MB
1 TByte = 1024 GB

**RAM (Random Access Memory)**
→ Wahlfreier Zugriff
→ Zugriffe dauern alle gleich lang
→ Lesen und Schreiben Möglich

**SRAM (Statisches RAM)**
→ statischer RAM-Speicher

**DRAM (Dynamisches RAM)**
→ Dynamischer RAM-Speicher

Informationen gehen nach Herausnahme verloren

**ROM (Read Only Memory)**
→ Nur lesender Zugriff

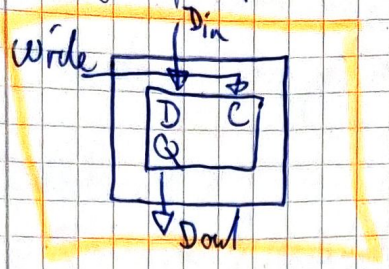
**PROM (Programmable Read Only Memory)**
→ einmalig beschreibbares ROM

Informationen bleiben über Jahre erhalten

# Technische Informatik

## SRAM

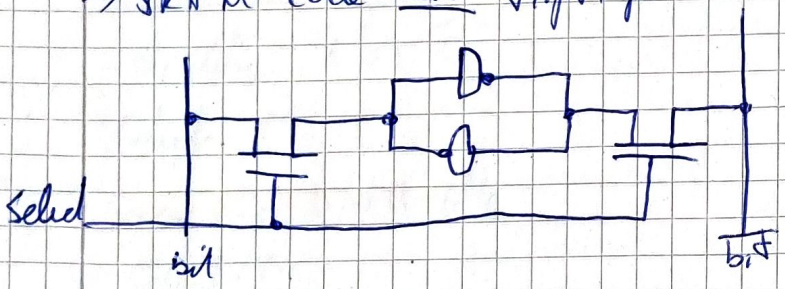
- Auf Flipflop Prinzip basierender Speicher




→ Bei Write = 1 wird Information von $D_{in}$ gespeichert und steht am Ausgang Dout zur Verfügung

→ Information bleibt gespeichert, auch wenn sich Din bei Write = 0 wieder ändern sollte

→ SRAM-Zelle kein Flipflop




Zellen sind reihen- und spaltenweise angeordnet

→ schneller Zugriff

→ Großer Platzbedarf

→ Großer Energiebedarf bei aktiven Zustand

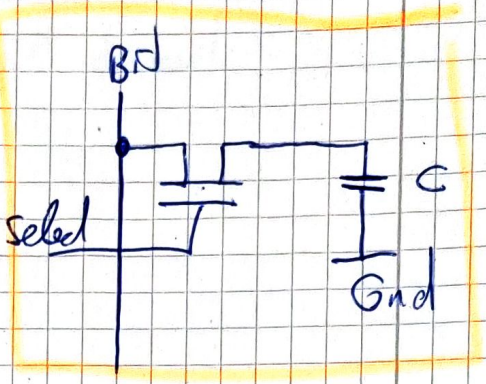
→ Unempfindlich gegen Strahlung und Erschütterung

# Technische Informatik

## DRAM

- Speicher basiert auf in einem Kondensator gespeicherter Ladung
- Speicher muss periodisch erneuert werden

DRAM Zelle:




- Transistor & Kondensator
- Kondensator zur Speicherung der Information
- Eine Schalt-/Leseleitung pro Spalte
- Eine Auswahlleitung pro Zeile diese schaltet Transistor

### Schreiben einer DRAM Zelle

- Bit Leitung "1" or "0"
- Select "1"

### Lesen einer DRAM-Zelle

- Select "1"
- Ladung des Kondensators über Bit-Leitung messen
- Lesevorgang zerstört Ladung

### Auffrischung

- Periodisches Lesen der DRAM-Zelle gleicht Ladungsverlust wieder aus

→ Steuerleitungen RAS/CAS codiert (Row Address Strobe / Column Address Strobe

### Zugriff auf DRAM

1) - Zeilendecoder liefert Select-Leitung für eine Zeile
   - Komplette Zeile auf Zwischenpuffer
2) - Bits auf Zwischenpuffer lesen

# Technische Informatik

## DRAM - Eigenschaften

- Weniger Platzbedarf
- Langsamer Zugriff
- Hoher Energieverbrauch sowohl bei Aktivität als auch bei Inaktivität

Technische Informatik – Blatt 11

## (11-1) SRAM und DRAM

a) Vergleichen Sie die Funktionsweise einer SRAM- und einer DRAM-Zelle in einem Workstation

→ Eine SRAM-Zelle ist ein Speicher für ein Bit, der seinen Wert behält solange die Stromversorgung sichergestellt ist. Die Speicherung des Bit erfolgt mittels zweier gegeneinander geschalteten Inverter. → Ein Eingang 1 oder 0

→ Eine SRAM-Zelle hat drei Anschlüsse: zwei Kanäle für den Eingang und Ausgang genutzt und eine Select-Leitung, die nur ein Eingang ist.

→ Um die Zelle auszuwählen, also zu lesen oder auch zu beschreiben muss man die Select-Leitung auf 1 setzen

→ SRAM-Zellen heute aus 6 Transistoren aufgebaut werden

→ Möchte man die Zelle beschreiben, legt man ein Bit und seinen Inverten Wert an je einen Eingang, der zum Inverter führt, an und setzt die Select Leitung auf 1

Technische Informatik – B [illegible]

(11-1)

a) – D Eine DRAM-Zelle speichert ebenfalls ein Bit; sie behält aber ihren Wert nicht automatisch, sondern muss ständig periodisch wieder aufgefrischt werden. Dies liegt daran, dass die Speicherung des Bits durch Ladung in einem Kondensator erfolgt der diese Ladung mit der Zeit durch ein Leckstrom verliert

⇒ Zur Auswahl der Zelle und Beschaltung des Kondensators ist auch bei der DRAM-Zelle ein Select-Leitung vorgesehen. Im Gegensatz zur SRAM-Zelle ist für die DRAM-Zelle nur eine weitere Leitung für die I/O-Leitung nötig

b) Stellen Sie die Vor- & Nachteile eines SRAM/DRAM gegenüber

| SRAM | DRAM |
|---|---|
| (+) Schneller Zugriff | (–) Langsamer Zugriff |
| (–) Großer Energieverbrauch im aktiven Zustand | (+)(–) Hoher Energieverbrauch im aktiven & passiven Zustand |
| (+) Kein Energieverbrauch im passiven Zustand | |
| (–) Großer Platzbedarf | (+) Geringer Platzbedarf |
| (–) Teuer | (+) Günstiger |
| (+) Unempfindlich gegen Störungen | |

# Technische Informatik Blatt 11

**A1-2)** SRAM-Zelle

64 Byte / 32 Byte mit 4 Byte word Länge

# 1:2 Demultiplexer $n_{Dem}$:

$$n_{Dem}^{32B} = \text{Zeilen} - 1 = \frac{32\,B}{4\,B} - 1 = 7$$

$$n_{Dem}^{64B} = \text{Zeilen} - 1 = \frac{64\,B}{4\,B} - 1 = 15$$

# 2:1 Multiplexer $n_{Mul}$:

$$n_{Mul}^{32B} = (\text{Zeilen} - 1)\,\text{Spalten} = 7 \cdot 32 = 224$$

$$n_{Mul}^{64B} = (\text{Zeilen} - 1)\,\text{Spalten} = 480$$

Technische Informatik

## Virtueller Speicher

Entkopplung des Speicherbedarf von verfügbaren Hauptspeicher

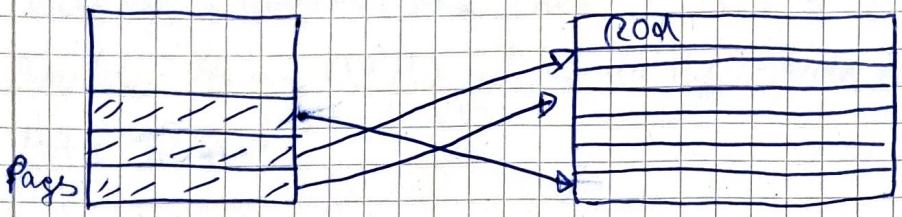
→ Prozesse benötigen nicht alle Speicherstellen gleich häufig

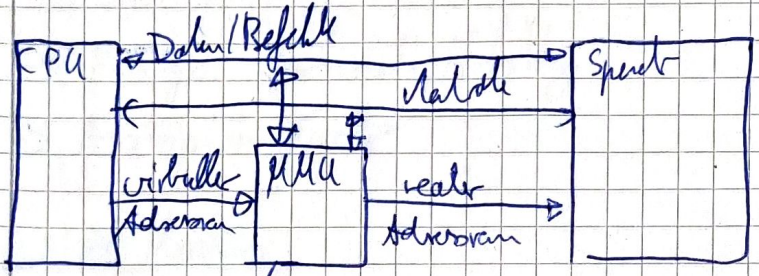
→ Prozesse benötigen evtl. mehr Speicher als Hauptspeicher vorhanden

## Paging

### Entstehung des virtuellen Adressraum

Virtueller Adressraum: Realer Adressraum:




→ heiße MMU a Translater

### Page Tabelle

→ Kleine Seite evtl. nicht vollständig genutzt

→ Kleine Seite benötigt interne Fragmentierung

→ Große Tabelle

# Reelische Hierarchie

## Translation Look-Aside Buffer (TLBs)

### Seitentabellen im Hauptspeicher

- Jeder einzelne Zugriff auf ein Befehl oder ein Datum erfordert zusätzliche Speicherzugriffe.

=> Unakzeptables Verlangsamung der Ausführung

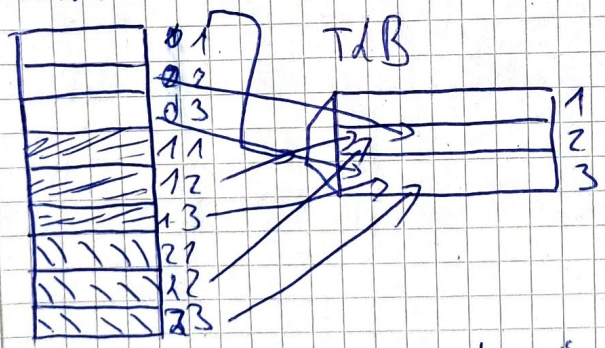
=> Einführung zusätzlicher, kleiner, schneller Hardwarespeicher, welche die häufig benötigten Tabelleneinträge enthalten

Diese heißen Translation Look-Aside Buffer oder auch Assoziative Translation Memory (ATM) und sind als Pufferspeicher direkt in die MMU integriert

-> Direct Mapping
-> Mengen-Assoziative Abbildung
-> Assoziativspeicher

### Direct Mapping

Virtuelle Seitennummer



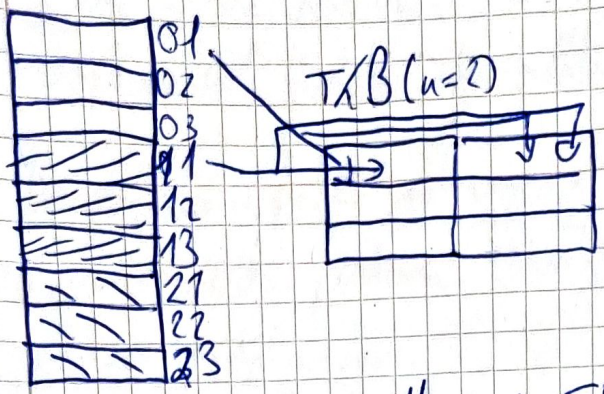

Alle Seiten, deren Adressen im Indexfeld übereinstimmen werden derselben TLB-Zeile zugeordnet

-> Führt zu starkem Anwachsen der Laufzeit

Technische Informatik

## Mengen-assoziative Abbildung

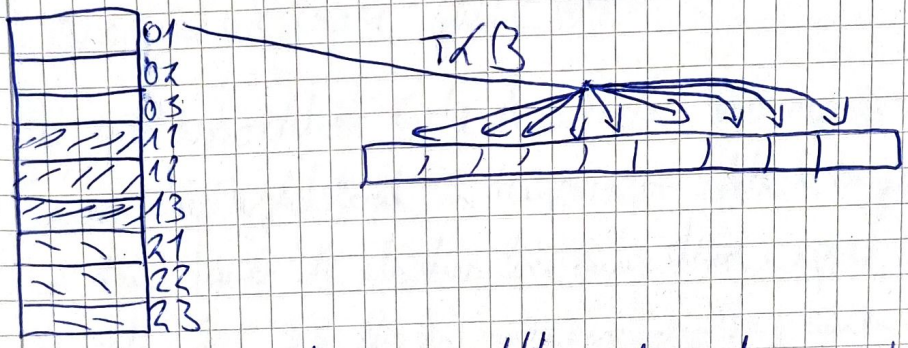
virtuelle Adressen




Bei jedem Index-Wert gibt es in TLB n zugeordnete Plätze mit n ≥ 1.
Sind alle n Plätze für einen neuen zuzuordnenden Index-Wert belegt, wird ein Platz im TLB überschrieben.

## Assoziativspeicher - assoziative mapping

virtuelle Spalt




Der Fall associate mapping entfällt aus dem set-associative mapping.
Jeder Platz des TLB hat die Verwaltungsinfos für jede beliebige Seite enthalten

# Technische Informatik - Blatt 12

## (12-1) Cache - Grundlagen

a) Auf welche Art kann Prozessor-Caches die Programmausführung optimieren? Welche Daten sollten im Cache liegen?

Hauptspeicherzugriffe sind langsam ⇒ wichtige Daten im Caches zwischenspeichern. Wichtig sind die Daten, die als nächstes benötigt werden.
Häufige Annahme ist, dass die folgenden Daten demnächst gebraucht werden:
→ Zuletzt häufig gebraucht
→ Nahe bei den zuletzt gebrauchten Daten liegen

b) Wäre ein direct mapping Cache günstiger als ein full associative Cache gleicher Größe?

Ein full associative Cache ist in diesem Fall teurer, weil hierbei die Vergleichslogik im Komparator erheblich komplexer ist damit teurer ist als dies bei einem direct mapped Cache der Fall wäre. Bei Abfrage muss nur der Tag mit einem Wert im Cache verglichen werden und nicht mit allen gleichzeitig.

# Technische Informatik Blatt 12

(12.2)

a) Geben Sie die Länge in Bit der drei Cache-Parameter Tag, Index und Offset für ein Direct Mapped Cache mit 4 KiB Nutzdaten Größe an. Der Rechner in dem der jeweilige Cache verbaut sei, verwendet für die Adressierung 32 Bit-Adressen. Die kleinste adressierbare Einheit des Rechners sei 1 Byte. Eine Cacheline fasse 32 Bit

$\frac{4\ \text{KiB}}{32\ \text{Byte}} = 128$ Cachelines $\Rightarrow$ 7 Index Bits

5 Bit Offset & Tag 20 Bits?